

Acceleration of Full-Batch Algorithms for Finite-Sum Problems through Parallel Computing

Marco Trambusti
Matricola - 7135350

marco.trambusti1@edu.unifi.it

Abstract

*The purpose of this work is to analyze full-batch algorithms for finite-sum problems by implementing **logistic regression** with L2 regularization in Python, including methods for dataset loading and execution of Gradient Descent with Armijo line search and Conjugate Gradient with Wolfe line search, while monitoring execution times and iteration counts. An initial sequential version was parallelized, analyzing acceleration as a function of the number of threads used and dataset size. Parallelization focuses on the computation of the loss function and its gradient.*

1. Introduction

Finite-sum optimization problems frequently arise in machine learning, as in many supervised learning tasks where the loss function to minimize is composed of a sum of terms, each corresponding to the error on a single data sample.

Definition – Finite-sum problems:

$$\min_{x \in \mathbb{R}^n} f(x) = \frac{1}{N} \sum_{i=1}^N f_i(x) \quad (1)$$

An important example is training **logistic regression models**, which can be formulated as:

$$\min_{w \in \mathbb{R}^p} \mathcal{L}(w) + \lambda \Omega(w) \quad (2)$$

where the loss function is defined as:

$$\mathcal{L}(w; X, y) = \sum_{i=1}^n \log(1 + \exp(-y^{(i)} w^T x^{(i)})) \quad (3)$$

This is an **unconstrained nonlinear optimization problem**, which can in principle be solved using standard methods. In this work, the methods considered are **Gradient Descent** with Armijo line search and **Conjugate Gradient** with Wolfe line search, both guaranteeing convergence to optimal solutions.

Given the structure of the problem, it is advantageous to exploit the possibility of **computing the objective function and its gradient in a distributed manner**. Since each term in the objective function is independent, evaluations of the function and gradient can naturally be parallelized across multiple cores or compute nodes.

1.1. Logistic Regression with L2 Regularization

The logistic regression model with L2 regularization is:

$$\mathcal{L}_{log}(w) + \lambda \Omega(w) = \sum_{i=1}^n \log(1 + \exp(-y^{(i)} w^T x^{(i)})) + \frac{\lambda}{2} \sum_{j=1}^d w_j^2 \quad (4)$$

The regularizer is necessary to prevent phenomena such as overfitting, which are detrimental to learning the function. The gradient of the loss is:

$$\nabla_w \mathcal{L}(w; X, y) = X^T r + \lambda w \quad (5)$$

with residual vector $r \in \mathbb{R}^n$, where

$$r_i = -y^{(i)} \sigma(-y^{(i)} w^T x^{(i)}), \quad \forall i = 1, \dots, n. \quad (6)$$

1.2. Gradient Descent with Armijo Line Search

Gradient Descent is an iterative method that minimizes an objective function, in this case the loss function, by updating the weight vector $\bar{w}$ at each iteration based on a step size α_k . The update rule is:

$$w_{k+1} = w_k - \alpha_k \nabla f(w_k) \quad (7)$$

where α_k is chosen using Armijo backtracking:

$$f(w_k - \alpha \nabla f(w_k)) \leq f(w_k) - \gamma \alpha \|\nabla f(w_k)\|^2 \quad (8)$$

As illustrated in the pseudocode below, the gradient method iterates the steps until the stopping condition $\nabla f(x^k) = 0$ is reached, or practically a sufficiently small value, e.g., 10^{-4} .

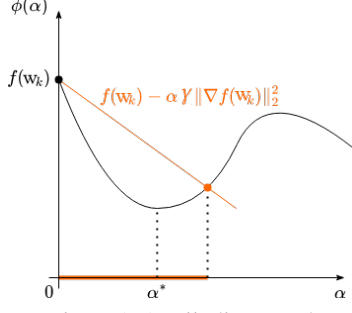


Figure 1. Armijo line search

Algorithm 1 Gradient Descent with Armijo Line Search

```

 $w \leftarrow w_0 \in \mathbb{R}^n; k \leftarrow 0$ 
while  $\nabla f(w^k) \neq 0$  do
   $d_k \leftarrow -\nabla f(w^k)$ 
  Compute  $\alpha_k$  along  $d_k$  using Armijo
   $w^{k+1} \leftarrow w^k - \alpha_k \nabla f(w^k)$ 
   $k \leftarrow k + 1$ 
end while

```

1.3. Conjugate Gradient with Wolfe Line Search

The Conjugate Gradient method, also iterative, improves convergence by combining gradient information with previous search directions:

$$d_{k+1} = -\nabla f(w_{k+1}) + \beta_k d_k \quad (9)$$

with Fletcher-Reeves update:

$$\beta_k = \frac{\|\nabla f(w_{k+1})\|^2}{\|\nabla f(w_k)\|^2} \quad (10)$$

The step size α_k is chosen to satisfy Wolfe conditions:

$$f(w_k + \alpha_k d_k) \leq f(w_k) + \gamma \alpha_k \nabla f(w_k)^T d_k \quad (11)$$

$$|\nabla f(w_k + \alpha_k d_k)^T d_k| \leq \sigma |\nabla f(w_k)^T d_k| \quad (12)$$

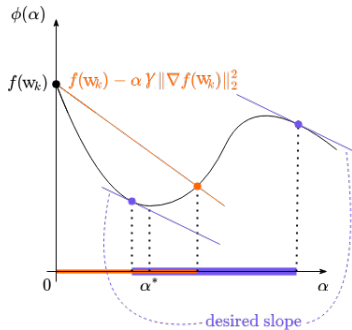


Figure 2. Wolfe conditions

The following pseudocode illustrates the Conjugate Gradient method with Wolfe line search:

Algorithm 2 Conjugate Gradient with Wolfe Line Search

```

1:  $x^0 \in \mathbb{R}^n, d_0 \leftarrow -g_0$ 
2: while  $\|g_k\| > \varepsilon$  do
3:   Compute  $\alpha_k$  using Wolfe
4:    $x^{k+1} \leftarrow x^k + \alpha_k d_k$ 
5:    $g_{k+1} \leftarrow \nabla f(x^{k+1})$ 
6:    $\beta_{k+1} \leftarrow \frac{\|g_{k+1}\|^2}{\|g_k\|^2}$ 
7:    $d_{k+1} \leftarrow -g_{k+1} + \beta_{k+1} d_k$ 
8: end while

```

1.4. Remarks

Gradient Descent is robust and simple to implement but tends to converge slowly, especially in the presence of functions with widely varying curvature along different directions. This makes it less efficient in high-dimensional scenarios or with unfavorable numerical conditions.

In contrast, the Conjugate Gradient method leverages deeper information about the problem structure, enabling faster convergence, particularly for quadratic and well-conditioned problems. However, this greater efficiency comes at the cost of more complex line search, requiring careful management of parameters and descent directions. In both cases, the ability to parallelize operations—such as gradient computation or objective function evaluation—is a key factor for applicability to large datasets typical in machine learning. Computational efficiency therefore depends not only on algorithm choice but also on its ability to exploit available hardware resources.

2. Implementation

The implementation was developed in Python, using the NumPy library to efficiently perform linear algebra operations. However, summations were computed manually instead of using `numpy.sum()` to avoid automatic parallelization introduced by underlying libraries such as OpenBLAS, which could interfere with control over the computational flow.

For optimizing the regularization parameter λ , cross-validation was employed using the `LogisticRegression` class from `scikit-learn`, configured with L2 regularization. The optimal λ values obtained were stored in the file `cv_results.csv`, making them easily reusable for subsequent experimental sessions or model comparisons.

2.1. Data Representation

Data is loaded from files in the `Datasets` folder using the `load_data` method, which relies on the `load_svmlight_file` function. This function imports the feature matrix X in dense format, where each row rep-

resents a sample and each column a descriptive variable. Labels y are loaded into a vector, encoded as binary values belonging to the set $\{-1, 1\}$.

If necessary, the matrix X is normalized to ensure consistent scaling across features. Finally, both X and y are converted to `float32` to optimize computational efficiency for subsequent processing and GPU parallelization.

2.2. Parameters

In `main.py`, key parameters are initialized to ensure proper functioning of line-search strategies according to Armijo and Wolfe criteria. The main ones are:

- **Regularization parameter** λ : determined via cross-validation.
- **Tolerance**: set to 10^{-4} , used as the stopping criterion for convergence.
- **Armijo parameters**: $\gamma = 10^{-4}$, $\delta = 0.5$, where γ controls sufficient decrease and δ adjusts step reduction.
- **Wolfe parameters**: $\gamma = 10^{-4}$, $\delta = 0.8$, $\sigma = 0.2$, with σ imposing a curvature condition to ensure descent direction quality.

2.3. Sequential LogRegressionL2

The `LogisticRegressionL2` class in `log_regression_l2.py` implements logistic regression with L2 regularization. At each iteration, both the objective function (loss) and its gradient are computed using explicit summation operations to avoid automatic parallelization by libraries like OpenBLAS.

Loss computation is handled by the `compute_loss` function, which includes the regularization term:

```
1 def compute_loss(self, X, y, w):
2     ...
3     logits = y * np.dot(X, w)
4     loss_terms = np.logaddexp(0, -logits)
5     total_loss = 0.0
6     for i in range(len(loss_terms)):
7         total_loss += loss_terms[i]
8
9     reg = 0.0
10    for wi in w:
11        reg += wi * wi
12    regularization = (self.lambda_reg / 2) * reg
13    loss = total_loss + regularization
14    ...
```

The gradient is computed in `compute_loss_gradient`, implementing the derivative of the regularized logistic loss:

```
1 def compute_loss_gradient(self, X, y, w):
2     ...
3     logits = X.dot(w)
4     z = -y * logits
5     sig = self.sigmoid(z)
6     r = -y * sig
7     grad = X.T.dot(r) + self.lambda_reg * w
8     ...
```

Line searches based on Armijo and Wolfe criteria are implemented as dedicated methods (`armijo_line_search` and `wolfe_line_search`). Both methods update weights w only if sufficient decrease (Armijo) or curvature (Wolfe) conditions are satisfied.

Execution time for loss and gradient computation is monitored separately and accumulated in `self.total_time`, useful for performance analysis.

2.3.1 Heuristic for Initial Step Size

In the `armijo_line_search` method, the initial step size Δ_0 is determined heuristically to ensure a good balance between numerical stability and convergence speed. The adopted formula is:

$$\Delta_0 = \min(\bar{\alpha}, \frac{1}{\delta} \cdot \alpha_{k-1}) \quad (13)$$

where:

- $\bar{\alpha}$ is the maximum allowed step size (default is 1),
- $\delta \in (0, 1)$ is the backtracking reduction factor (typically 0.5),
- α_{k-1} is the step used in the previous iteration.

This choice allows reusing historical information about the objective function trend, avoiding restarting from excessively large values that could cause instability.

The code snippet implementing this logic is as follows:

```
1 delta_0 = min(alpha_max, alpha_prev / delta)
2 alpha = delta_0

During the search, the step is progressively reduced until the Armijo condition is satisfied:

1 if loss_new
2     <= current_loss + gamma * alpha * np.dot(grad, direction):
3     break
4 alpha *= delta
```

This heuristic proved effective in improving the stability of the gradient descent method, especially in the presence of objective functions with varying curvature.

2.4. Conjugate Gradient with Wolfe

The Conjugate Gradient method implemented in the `LogisticRegressionL2` class achieves faster convergence than standard gradient descent by leveraging previous directions to build more effective descent paths. At each iteration, the direction is updated using the Fletcher-Reeves formula.

Line search is performed using Wolfe conditions, ensuring both sufficient decrease of the objective function and adequate curvature along the descent direction. The step size α_k is accepted only if both conditions are met.

```

1 if (loss_new
2     <= current_loss+gamma*alpha*np.dot(grad, direction)
3     and np.abs(np.dot(grad_new, direction))
4     <= sigma*np.abs(np.dot(grad, direction))):
5     break
6     break

```

The core loop of the conjugate gradient method is implemented as follows:

```

1 lr = self.wolfe_line_search(grad, X, y, direction)
2 self.w = np.add(self.w, lr * direction)
3 grad_new = self.compute_loss_gradient(X, y, self.w)
4 beta = np.dot(grad_new, grad_new) / np.dot(grad, grad)
5 direction = -grad_new + beta * direction

```

The `conjugate_gradient_wolfe` method terminates when the gradient norm falls below the predefined tolerance threshold.

3. Parallelization

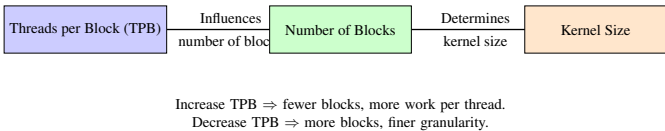
3.1. Parallelization Points

To accelerate L2 logistic regression, a parallel version was implemented using Numba CUDA, which allows writing GPU kernels directly in Python. The most expensive operations, such as loss and gradient computation, were split into CUDA kernels that exploit the independent nature of the data: each thread processes a sample or a feature, while partial results are aggregated through hierarchical reductions.

The gradient is computed in two steps: (i) computation of the intermediate vector $r_i = -y_i \cdot \sigma(-y_i x_i^T w)$ for each sample, (ii) multiplication $X^T r$ in parallel, assigning each thread to a feature. L2 regularization (λw) is handled on the CPU to avoid data transfer overhead.

An important aspect is that the effective kernel size varies based on the chosen `threadsperblock` (TPB) parameter: increasing TPB reduces the number of blocks required, while lower values increase parallelism granularity. This balance is crucial to fully utilize GPU resources and minimize idle time.

3.2. Kernel Size Diagram



3.3. Warmup: Pre-Initialization

To reduce latency caused by JIT compilation of kernels, a *warmup* method was introduced that performs artificial calls on dummy data. This ensures kernels are compiled and loaded before real executions, avoiding delays and enabling accurate performance measurements.

4. Results and Performance Evaluation

4.1. Setup

Tests were conducted on a machine with Intel Core i7-6700K CPU (4 cores, 8 threads) and NVIDIA GeForce GTX 970 GPU (1664 CUDA cores, 4GB memory). Datasets from this site of increasing size were used to assess scalability. The sequential version was compared with parallel versions using 32, 64, 128, 256, 512 & 1024 threads. To validate the optimization methods, results were compared with `scikit-learn`'s Logistic Regression using L2 regularization. Both Gradient Descent and Conjugate Gradient produced consistent final loss values and gradient norms, confirming correctness.

4.2. Results and Evaluation

In sequential version, Conjugate Gradient generally requires fewer iterations than Gradient Descent, especially for larger datasets. However, for smaller datasets (e.g., 1000×24), GD is faster despite more iterations, indicating that CG's overhead is not always beneficial for low-dimensional problems. Overall, CG demonstrates superior scalability and efficiency for large-scale optimization tasks.

Dataset	GD Time (s)	GD It.	CG Time (s)	CG It.
768×8	4.91	766	2.19	41
1000×24	2.88	391	6.20	67
1000×60	0.27	30	0.98	11
1605×119	87.19	3949	51.01	204
3185×122	477.03	7730	336.25	336
11055×68	670.30	4298	380.74	287

Table 1. Execution time and iterations for GD and CG (sequential)

Execution time analysis showed significant improvements in the parallel version, with differences depending on dataset size and dimensionality.

Method	Dataset	TPB	Sp. Total	Time (s)	It.
GD	768×8	256	0.83×	5.88	766
CG	768×8	256	0.78×	2.81	44
GD	1000×24	256	0.96×	3.00	391
CG	1000×24	256	0.98×	6.33	67
GD	1000×60	256	0.93×	0.29	30
CG	1000×60	256	1.05×	0.94	11
GD	1605×119	512	1.47×	59.30	3949
CG	1605×119	512	1.86×	27.49	149
GD	3185×122	512	3.72×	128.12	7730
CG	3185×122	512	5.05×	66.56	289
GD	11055×68	32	13.76×	48.72	4298
CG	11055×68	32	9.07×	41.98	336

Table 2. Configurations with best total speedup (GD and CG)

Small datasets (768 samples, 8 features). For small-scale problems, parallelization overhead is evident. Gradient Descent (GD) with Armijo line search required 766 iterations and about 4.91 seconds, while Conjugate Gradient (CG) with Wolfe line search converged in only 44 iterations and about 2.81 seconds. CUDA parallelization did not provide substantial benefits: best-case speedups were marginal ($S_p \approx 0.83$ for GD, 0.78 for CG), and efficiency dropped sharply due to synchronization costs. This confirms that GPU acceleration is not advantageous for small-scale problems.

Medium datasets (1000 samples, 24–60 features). As dimensionality increases, parallelization benefits become more evident. For 24 features, sequential GD took 2.88 s, while CUDA with 256 threads reduced time to 3.00 s ($S_p = 0.96$). Sequential CG took 6.20 s, while CUDA with 256 threads achieved 6.33 s ($S_p = 0.98$). For 60 features, sequential GD was very fast (0.27 s), and CUDA versions achieved similar times, with speedups close to 1. Sequential CG required 0.98 s, while CUDA reduced it to about 0.94 s with 256 threads. In this regime, GPU advantage is modest, but CG converges in fewer iterations than GD.

Large datasets (1605–3185 samples, 119–122 features). Here, parallelization shows clear benefits. For 1605 samples and 119 features, sequential GD took 87.19 s, while CUDA with 512 threads reduced it to 59.30 s ($S_p = 1.47$). Sequential CG took 51.01 s, while CUDA with 512 threads reduced it to 27.49 s ($S_p = 1.86$). Similarly, for 3185 samples and 122 features, sequential GD took 477.03 s, while CUDA with 512 threads reduced it to 128.12 s ($S_p = 3.72$). Sequential CG took 336.25 s, while CUDA with 512 threads achieved 66.56 s ($S_p = 5.05$). These results confirm the effectiveness of GPU acceleration for large-scale problems.

Very large dataset (11,055 samples, 68 features). In this case, GPU advantage is remarkable. Sequential GD took 670.30 s, while CUDA with 32 threads reduced it to 48.72 s ($S_p = 13.76$). Sequential CG took 380.74 s, while CUDA with 32 threads reduced it to 41.98 s ($S_p = 9.07$). Both methods achieved excellent convergence, with final loss values consistent across implementations. However, efficiency decreased with very high thread counts (512–1024) due to memory bandwidth saturation and synchronization costs.

5. Results Analysis

5.1. Computational Performance

5.1.1 Execution Time

Total execution time grows linearly in sequential methods, while parallel versions exhibit more favorable scalability.

For small datasets, initialization and synchronization overhead can outweigh benefits, but for large datasets GPU acceleration is significant.

For example, for the dataset with $11,055 \times 68$:

- Sequential GD: 670.30 s
- GD CUDA (TPB=32): 48.72 s
- Speedup: $13.76\times$

5.1.2 Speedup

Speedup is defined as:

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}}$$

and was calculated for three components: total time, gradient computation, and loss computation. Loss computation benefits most from parallelization, with speedups up to $18.06\times$.

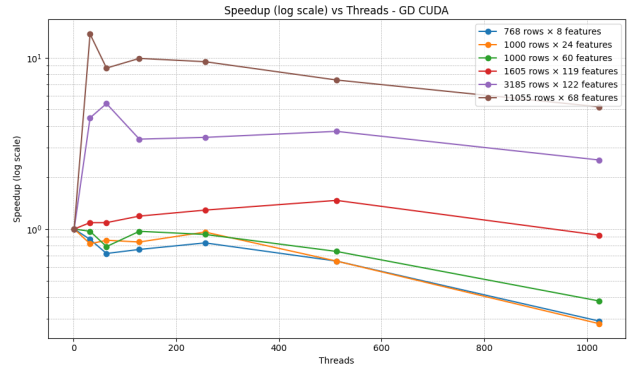


Figure 3. Gradient Descent Total speedup vs TPB

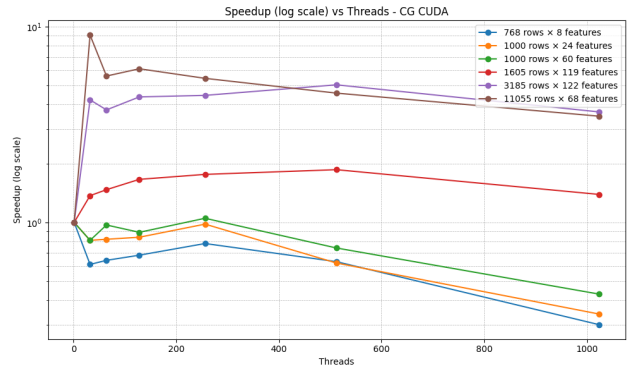


Figure 4. Conjugate Gradient Total speedup vs TPB

5.2. Convergence and Accuracy

5.2.1 Iterations

The number of iterations required for convergence differs significantly between GD and CG. CG requires far fewer iterations (e.g., 336 vs 4298 for 11,055 *times*68), but each iteration is more expensive due to Wolfe line search.

5.2.2 Loss and Gradient Norm

All methods converge to similar loss values (differences $< 10^{-4}$) and gradient norms $\approx 10^{-4}$, indicating numerical stability.

L2 regularization contributed to avoiding divergence.

6. Conclusions

This study examined the impact of optimization methods and parallelization on machine learning performance. Conjugate Gradient proved more efficient than Gradient Descent, requiring significantly fewer iterations to converge, which makes it well-suited for large-scale problems.

The integration of Numba CUDA enabled substantial acceleration, with speedups up to $13.76\times$ overall. Gradient and loss computations benefited differently from parallelization, with the loss kernel showing the highest scalability due to its sample-wise independence.

Overall, combining advanced optimization techniques with GPU-based parallel programming offers a powerful strategy for scaling machine learning models efficiently.